PYTHON PROGRAMMING

ACTIVITY-BASED LEARNING

10 CASE STUDIES WITH COMPLETE SOLUTIONS

Student Name:	Erla.Kavya Amrutha
Roll Number:	26000897
Branch:	CSM
Section:	1
Course:	Python Programming
Institution:	_____

Submitted as part of the Python Programming Activity-Based Learning assignment.

Prepared according to the selected case-study topics and the mandatory deliverables in the assignment instructions.

CONTENTS

Case 12: Attendance Recovery Computation

Case 13: ATM Transaction Sequence Simulation

Case 17: Student Performance Trend Analysis

Case 27: Advanced Numeric Pattern Generator

Case 28: Repeated Transaction Detection

Case 29: Password Policy Validation Engine

Case 31: String-Based Examination Answer Evaluation

Case 35: Text Compression Using Character Runs

Case 42: Student Marks Matrix Analysis

Case 49: Tuple-Based Product Catalogue Audit

Each case includes: problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output, and conclusion.

CASE STUDY 12: Attendance Recovery Computation

Problem Statement

Develop a Python program that calculates a student's current attendance percentage and determines the minimum number of consecutive classes the student must attend to reach a required attendance threshold.

Objectives

• Calculate attendance percentage from classes attended and classes conducted.

• Validate attendance inputs and the required percentage.

• Use a loop to find the minimum additional classes needed when every future class is attended.

• Display the current status and recovery requirement.

Assumptions

• Classes attended and classes conducted are non-negative whole numbers.

• Classes attended cannot exceed classes conducted.

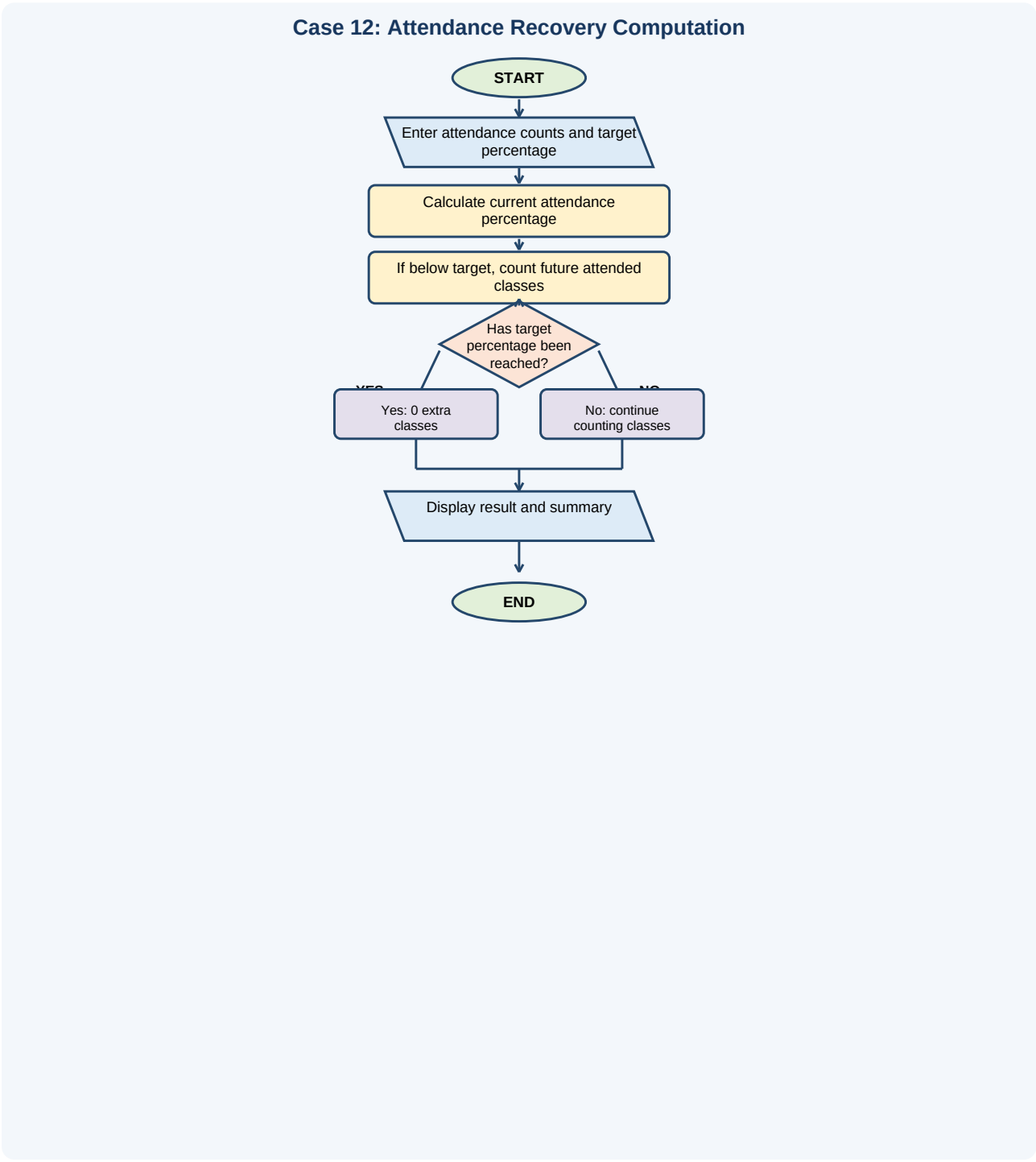
• The required attendance threshold is between 0 and 100 percent.

• For the recovery calculation, every additional class is attended.

Algorithm

1. Start the program.
2. Input classes attended, classes conducted, and the required attendance percentage.
3. Validate the values; reject invalid attendance counts or threshold values.
4. Calculate current attendance percentage (use 0 when no classes have been conducted).
5. If current attendance already meets the threshold, set additional classes to 0.
6. Otherwise, repeatedly add one attended class and one conducted class until the threshold is reached.
7. Display the current percentage, eligibility status, and minimum additional classes.
8. Stop.

CASE 12 – FLOWCHART



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 12 – PSEUDOCODE

```
BEGIN
    INPUT attended, conducted, required
    IF attended < 0 OR conducted < 0 OR attended > conducted
       OR required < 0 OR required > 100 THEN
        PRINT "Invalid input"
    ELSE
        IF conducted = 0 THEN
            current = 0
        ELSE
            current = attended * 100 / conducted
        END IF
        extra = 0
        future_attended = attended
        future_conducted = conducted
        WHILE future_attended * 100 / MAX(1, future_conducted) < required
              AND future_conducted > 0 OR (conducted = 0 AND required > 0)
            future_attended = future_attended + 1
            future_conducted = future_conducted + 1
            extra = extra + 1
        END WHILE
        PRINT current, extra
    END IF
END
```

Python Source Code

```
attended = int(input("Classes attended: "))
conducted = int(input("Classes conducted: "))
required = float(input("Required attendance (%): "))

if attended < 0 or conducted < 0 or attended > conducted or required < 0 or required > 100:
    print("Invalid input")
else:
    if conducted == 0:
        current = 0
    else:
        current = attended * 100 / conducted

    extra = 0
    future_attended = attended
    future_conducted = conducted

    while (future_conducted == 0 and required > 0) or (
        future_conducted > 0 and future_attended * 100 / future_conducted < required
    ):
        future_attended += 1
        future_conducted += 1
        extra += 1

    print("Current attendance =", round(current, 2), "%")
    if current >= required:
        print("Status: Eligible")
        print("Additional classes required = 0")
    else:
        print("Status: Attendance recovery needed")
        print("Minimum consecutive classes to attend =", extra)
```

CASE 12 – TESTING AND RESULT

Test Cases

Attended	Conducted	Required	Expected result
36	40	75	90%; already eligible; 0 classes
28	40	75	70%; 8 additional classes
0	0	75	0%; 1 additional class
45	50	90	90%; already eligible; 0 classes
12	20	80	60%; 20 additional classes

Sample Output

```
Classes attended: 28
Classes conducted: 40
Required attendance (%): 75
Current attendance = 70.0 %
Status: Attendance recovery needed
Minimum consecutive classes to attend = 8
```

Conclusion

The program calculates attendance percentage and uses a while loop to determine the smallest number of future classes that must all be attended to reach the required threshold.

CASE STUDY 13: ATM Transaction Sequence Simulation

Problem Statement

Create a Python program that processes a sequence of deposits and withdrawals, maintains a simulated account balance, rejects invalid transactions, applies a withdrawal limit, and prints a transaction summary.

Objectives

• Represent transactions as tuples containing a transaction type and amount.

• Process multiple transactions using a loop and conditional statements.

• Prevent withdrawals that exceed the available balance or the per-transaction limit.

• Summarise accepted deposits, withdrawals, rejected transactions, and closing balance.

Assumptions

• The opening balance is a non-negative amount.

• Each transaction is represented by a type ('D' for deposit or 'W' for withdrawal) and a positive amount.

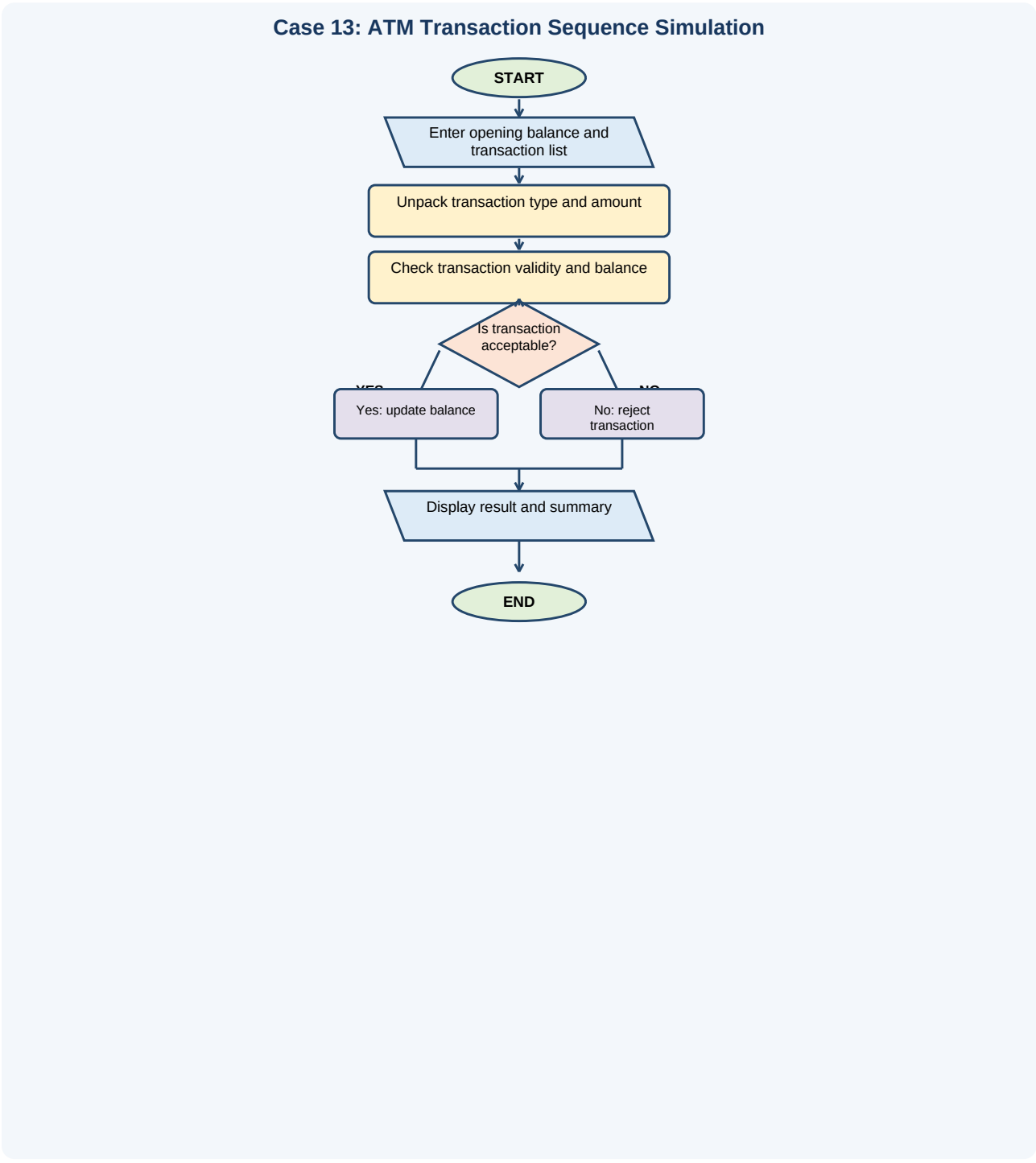
• The maximum permitted withdrawal per transaction is ■10,000.

• A withdrawal is accepted only when the amount does not exceed the balance and the withdrawal limit.

Algorithm

1. Start and input the opening balance.
2. Store the transaction sequence as a list of tuples.
3. Set deposit, withdrawal, and rejected-transaction totals to zero.
4. For each transaction, unpack its type and amount.
5. Reject non-positive amounts and unknown transaction types.
6. For a deposit, add the amount to the balance and deposit total.
7. For a withdrawal, check the limit and available balance before deducting it; otherwise count it as rejected.
8. Display the transaction summary and closing balance.
9. Stop.

CASE 13 – FLOWCHART



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 13 – PSEUDOCODE

```
BEGIN
    INPUT opening_balance
    transactions = [("D", 500), ("W", 200), ("W", 12000), ("W", 900)]
    balance = opening_balance
    deposits = 0
    withdrawals = 0
    rejected = 0
    FOR each transaction IN transactions
        UNPACK type, amount
        IF amount <= 0 THEN
            rejected = rejected + 1
        ELSE IF type = "D" THEN
            balance = balance + amount
            deposits = deposits + amount
        ELSE IF type = "W" THEN
            IF amount <= 10000 AND amount <= balance THEN
                balance = balance - amount
                withdrawals = withdrawals + amount
            ELSE
                rejected = rejected + 1
            END IF
        ELSE
            rejected = rejected + 1
        END IF
    END FOR
    PRINT deposits, withdrawals, rejected, balance
END
```

Python Source Code

```
opening_balance = float(input("Opening balance: ■"))
transactions = [("D", 500), ("W", 200), ("W", 12000), ("W", 900)]

balance = opening_balance
deposit_total = 0
withdrawal_total = 0
rejected = 0

for transaction in transactions:
    transaction_type, amount = transaction

    if amount <= 0:
        print("Rejected:", transaction)
        rejected += 1
    elif transaction_type == "D":
        balance += amount
        deposit_total += amount
        print("Deposit accepted:", amount)
    elif transaction_type == "W":
        if amount <= 10000 and amount <= balance:
            balance -= amount
            withdrawal_total += amount
            print("Withdrawal accepted:", amount)
        else:
            print("Withdrawal rejected:", amount)
            rejected += 1
    else:
        print("Invalid transaction:", transaction)
        rejected += 1

print("Total deposits = ■", deposit_total)
print("Total withdrawals = ■", withdrawal_total)
print("Rejected transactions =", rejected)
print("Closing balance = ■", balance)
```

CASE 13 – TESTING AND RESULT

Test Cases

Opening balance	Transactions	Expected closing balance	Rejected
■2,000	D500, W200, W900	■1,400	0
■1,000	W1,500	■1,000	1
■15,000	W12,000	■15,000	1
■500	D500, W900	■100	0

Sample Output

```
Opening balance: ■2000
Deposit accepted: 500
Withdrawal accepted: 200
Withdrawal rejected: 12000
Withdrawal accepted: 900
Total deposits = ■ 500
Total withdrawals = ■ 1100
Rejected transactions = 1
Closing balance = ■ 1400
```

Conclusion

The simulation demonstrates tuple unpacking, lists, loops, arithmetic operations, and conditional checks to process a sequence of ATM transactions safely within the stated rules.

CASE STUDY 17: Student Performance Trend Analysis

Problem Statement

Develop a Python program to compare a student's marks across multiple assessments, calculate improvement or decline, identify weak subjects, and classify the overall performance trend.

Objectives

• Store subject-wise marks across assessments using nested lists.

• Calculate each subject's change from the first assessment to the last assessment.

• Calculate subject averages and identify weak subjects using a defined threshold.

• Summarise whether the student's overall performance improved, declined, or remained unchanged.

Assumptions

• Marks are whole numbers from 0 to 100.

• The example uses three subjects and three assessments per subject.

• A subject average below 40 is treated as weak.

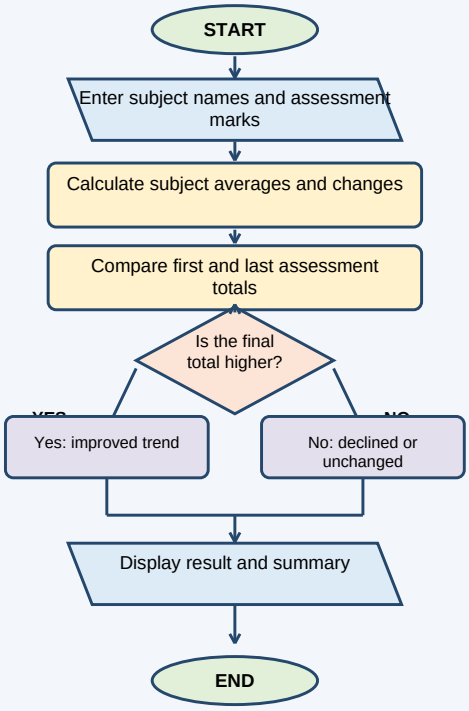
• Overall trend is based on the comparison of the total marks in the first and last assessments.

Algorithm

1. Start the program and define the subject names and marks matrix.
2. For each subject, calculate the average and the change between the first and last assessment.
3. Display the subject-wise average and trend.
4. Mark a subject as weak when its average is below 40.
5. Calculate the total marks for the first and last assessments.
6. Classify the overall trend as improved, declined, or unchanged.
7. Display weak subjects and the overall trend.
8. Stop.

CASE 17 – FLOWCHART

Case 17: Student Performance Trend Analysis



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 17 – PSEUDOCODE

```
BEGIN
    subjects = ["Python", "Mathematics", "English"]
    marks = [[45, 55, 70], [60, 58, 55], [30, 35, 40]]
    weak_subjects = []
    FOR i FROM 0 TO length(subjects)-1
        total = 0
        FOR j FROM 0 TO 2
            total = total + marks[i][j]
        END FOR
        average = total / 3
        change = marks[i][2] - marks[i][0]
        PRINT subjects[i], average, change
        IF average < 40 THEN
            ADD subjects[i] TO weak_subjects
        END IF
    END FOR
    first_total = marks[0][0] + marks[1][0] + marks[2][0]
    last_total = marks[0][2] + marks[1][2] + marks[2][2]
    IF last_total > first_total THEN
        trend = "Improved"
    ELSE IF last_total < first_total THEN
        trend = "Declined"
    ELSE
        trend = "Unchanged"
    END IF
    PRINT weak_subjects, trend
END
```

Python Source Code

```
subjects = ["Python", "Mathematics", "English"]
# Each row contains marks in Assessment 1, 2 and 3.
marks = [
    [45, 55, 70],
    [60, 58, 55],
    [30, 35, 40]
]

weak_subjects = []

print("SUBJECT-WISE PERFORMANCE")
for i in range(len(subjects)):
    total = 0
    for j in range(len(marks[i])):
        total += marks[i][j]

    average = total / len(marks[i])
    change = marks[i][-1] - marks[i][0]
    print(subjects[i], "Average:", round(average, 2), "Change:", change)

    if average < 40:
        weak_subjects.append(subjects[i])

first_total = 0
last_total = 0
for i in range(len(marks)):
    first_total += marks[i][0]
    last_total += marks[i][-1]

if last_total > first_total:
    trend = "Improved"
elif last_total < first_total:
    trend = "Declined"
else:
    trend = "Unchanged"

print("Weak subjects:", weak_subjects)
print("First assessment total:", first_total)
print("Last assessment total:", last_total)
print("Overall trend:", trend)
```

CASE 17 – TESTING AND RESULT

Test Cases

Subject marks (A1,A2,A3)	Average	Change	Expected classification
Python: 45,55,70	56.67	+25	Not weak
Mathematics: 60,58,55	57.67	-5	Not weak
English: 30,35,40	35.00	+10	Weak

Sample Output

```
SUBJECT-WISE PERFORMANCE
Python Average: 56.67 Change: 25
Mathematics Average: 57.67 Change: -5
English Average: 35.0 Change: 10
Weak subjects: ['English']
First assessment total: 135
Last assessment total: 165
Overall trend: Improved
```

Conclusion

The program uses nested lists and nested loops to analyse assessment marks, identify weak subjects, and compare overall performance across assessments.

CASE STUDY 27: Advanced Numeric Pattern Generator

Problem Statement

Write a Python program to generate a hollow diamond pattern using a user-specified size. The program must validate the input and align the pattern correctly using nested loops.

Objectives

• Accept and validate the size of the pattern.

• Use nested loops to generate the upper and lower halves of a hollow diamond.

• Print boundary symbols while leaving the interior hollow.

• Demonstrate loop-based pattern generation and formatted output.

Assumptions

• The size is a positive odd integer representing the total number of rows.

• The pattern uses the asterisk (*) character.

• The middle row contains the widest part of the diamond.

• Invalid even or non-positive sizes are rejected.

Algorithm

1. Start the program and input the number of rows.

2. Check whether the size is a positive odd integer.

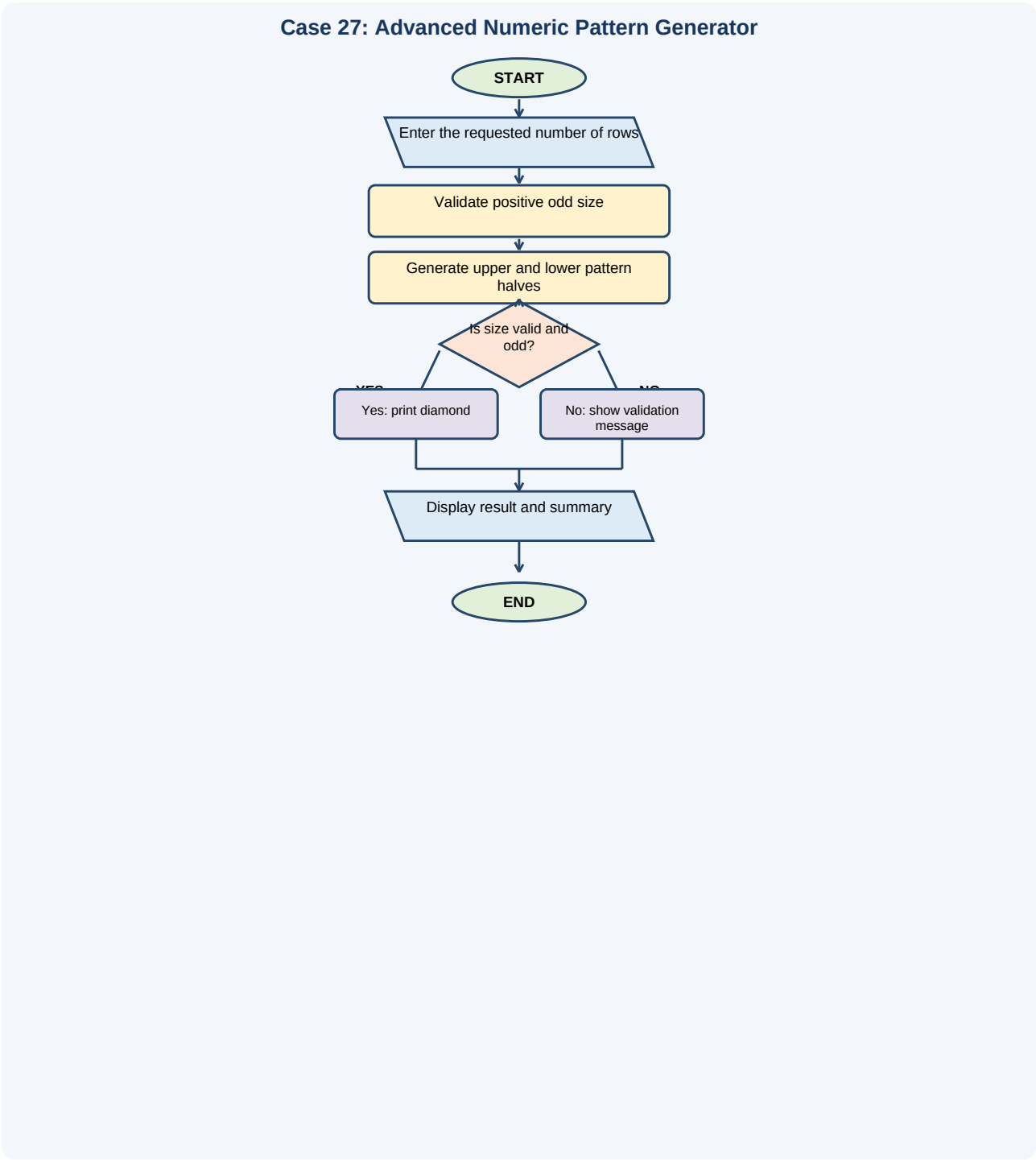
3. Generate the upper half including the middle row.

4. For each row, print leading spaces, the first star, inner spaces where needed, and the second star.

5. Generate the lower half in reverse order.

6. Display the hollow diamond and stop.

CASE 27 – FLOWCHART



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 27 – PSEUDOCODE

```
BEGIN
    INPUT n
    IF n <= 0 OR n MOD 2 = 0 THEN
        PRINT "Enter a positive odd number"
    ELSE
        middle = n // 2
        FOR row FROM 0 TO middle
            PRINT (middle-row) spaces
            IF row = 0 THEN
                PRINT "*"
            ELSE
                PRINT "*", (2*row-1) spaces, "*"
            END IF
        END FOR
        FOR row FROM middle-1 DOWNTO 0
            PRINT (middle-row) spaces
            IF row = 0 THEN
                PRINT "*"
            ELSE
                PRINT "*", (2*row-1) spaces, "*"
            END IF
        END FOR
    END IF
END
```

Python Source Code

```
n = int(input("Enter an odd number of rows: "))

if n <= 0 or n % 2 == 0:
    print("Please enter a positive odd number.")
else:
    middle = n // 2

    # Upper half, including the middle row
    for row in range(middle + 1):
        spaces = middle - row
        print(" " * spaces, end="")

        if row == 0:
            print("*")
        else:
            inner_spaces = 2 * row - 1
            print("*" + " " * inner_spaces + "*")

    # Lower half
    for row in range(middle - 1, -1, -1):
        spaces = middle - row
        print(" " * spaces, end="")

        if row == 0:
            print("*")
        else:
            inner_spaces = 2 * row - 1
            print("*" + " " * inner_spaces + "*")
```

CASE 27 – TESTING AND RESULT

Test Cases

Input size	Expected result
1	One star
3	Three-row hollow diamond
5	Five-row hollow diamond
4	Invalid: size must be odd
0	Invalid: size must be positive

Sample Output

```
Enter an odd number of rows: 5
  *
 * *
*   *
 * *
  *
```

Conclusion

The program generates a symmetric hollow diamond using nested loop logic, string repetition, and input validation.

CASE STUDY 28: Repeated Transaction Detection

Problem Statement

Create a Python program that examines a list of transaction identifiers and amounts, detects repeated identifiers, flags amounts above a defined threshold, and displays suspicious transactions.

Objectives

• Represent transaction records using tuples stored in a list.

• Compare transaction identifiers to detect duplicates.

• Identify transactions whose amounts exceed the specified threshold.

• Display the suspicious records and a summary count.

Assumptions

• Each transaction is a tuple containing an identifier and a positive amount.

• An identifier is considered repeated when it occurs more than once.

• The amount threshold in the sample is ■5,000.

• The program flags duplicate and high-value records; it does not determine whether a transaction is actually fraudulent.

Algorithm

1. Start and define the transaction list and amount threshold.

2. Create an empty list to store suspicious records.

3. Visit each transaction and compare its identifier with every other transaction.

4. If the same identifier occurs more than once, mark the record as repeated.

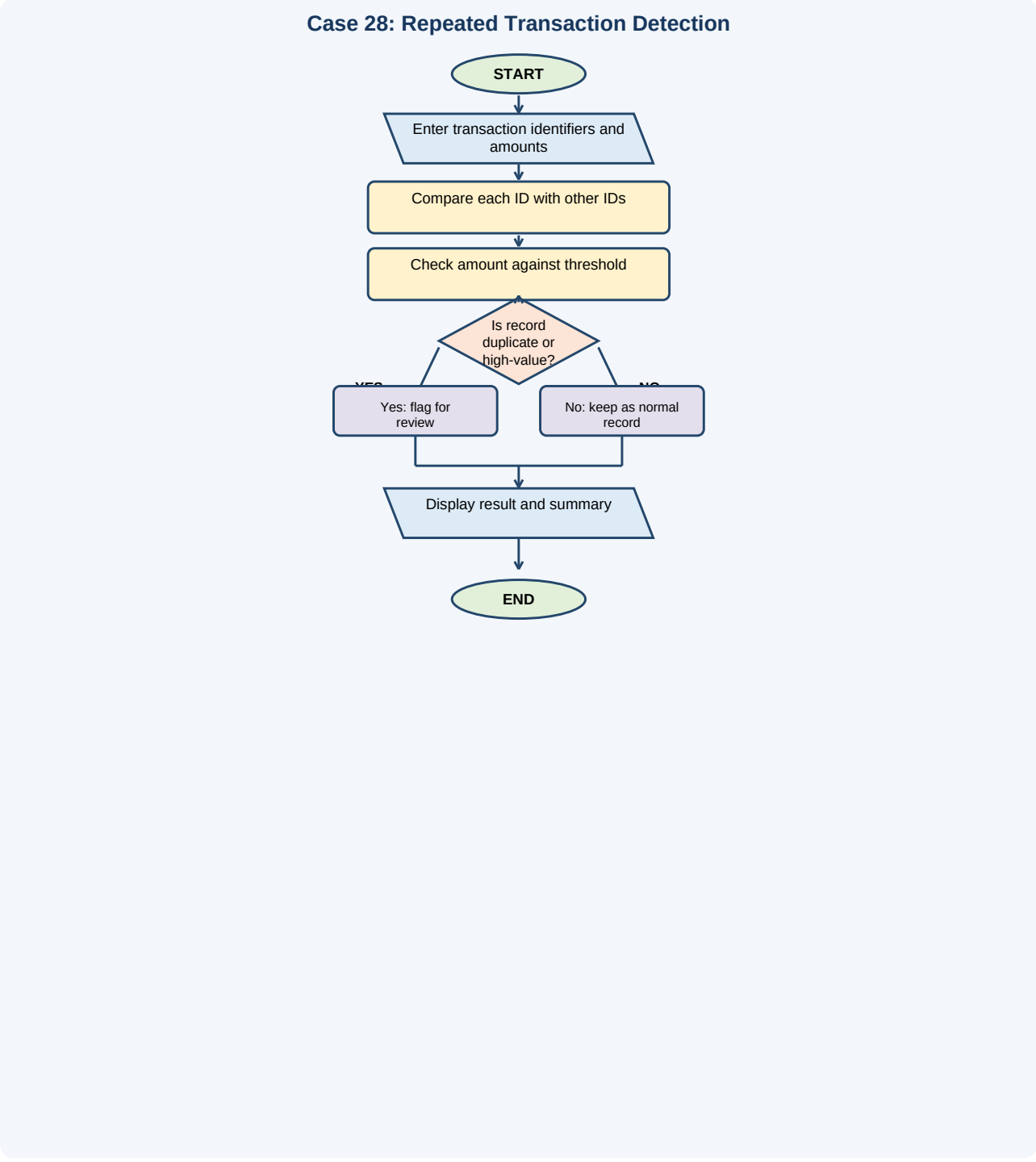
5. If the amount exceeds the threshold, mark the record as high value.

6. Add a record only once to the suspicious list.

7. Display suspicious records and the total number of flagged records.

8. Stop.

CASE 28 – FLOWCHART



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 28 – PSEUDOCODE

```
BEGIN
    transactions = [("T101", 2000), ("T102", 7000),
                    ("T101", 2000), ("T103", 900)]
    threshold = 5000
    suspicious = []
    FOR i FROM 0 TO length(transactions)-1
        id, amount = transactions[i]
        duplicate = FALSE
        FOR j FROM 0 TO length(transactions)-1
            IF i != j AND id = transactions[j].id THEN
                duplicate = TRUE
            END IF
        END FOR
        IF duplicate = TRUE OR amount > threshold THEN
            IF transactions[i] NOT IN suspicious THEN
                ADD transactions[i] TO suspicious
            END IF
        END IF
    END FOR
    PRINT suspicious
END
```

Python Source Code

```
transactions = [
    ("T101", 2000),
    ("T102", 7000),
    ("T101", 2000),
    ("T103", 900),
    ("T104", 6500)
]
threshold = 5000
suspicious = []

for i in range(len(transactions)):
    transaction_id, amount = transactions[i]
    duplicate = False

    for j in range(len(transactions)):
        other_id, other_amount = transactions[j]
        if i != j and transaction_id == other_id:
            duplicate = True

    high_value = amount > threshold

    if duplicate or high_value:
        if transactions[i] not in suspicious:
            suspicious.append(transactions[i])

print("Suspicious transactions:")
for record in suspicious:
    print(record)

print("Total flagged records =", len(suspicious))
```

CASE 28 – TESTING AND RESULT

Test Cases

Transaction records	Threshold	Expected flagged records
T1:■500, T2:■700	■5,000	None
T1:■2,000, T1:■2,000	■5,000	Both T1 records
T2:■7,000	■5,000	T2 high value
T3:■6,000, T3:■6,000	■5,000	T3 records, flagged once each

Sample Output

```
Suspicious transactions:
('T101', 2000)
('T102', 7000)
('T104', 6500)
Total flagged records = 3
```

Conclusion

The program uses lists, tuples, nested loops, and conditional statements to detect duplicate identifiers and high-value transactions for review.

CASE STUDY 29: Password Policy Validation Engine

Problem Statement

Develop a Python program that checks sample password strings against a set of password policy rules and reports which conditions are not satisfied.

Objectives

• Evaluate password strings using basic string operations and loops.

• Check minimum length and character-category requirements.

• Detect a prohibited substring and consecutive repeated characters.

• Report whether each sample password passes or fails and list the failed rules.

Assumptions

• A valid sample password must contain at least 8 characters.

• It must contain at least one uppercase letter, one lowercase letter, one digit, and one special character.

• The substring 'password' is prohibited without regard to letter case.

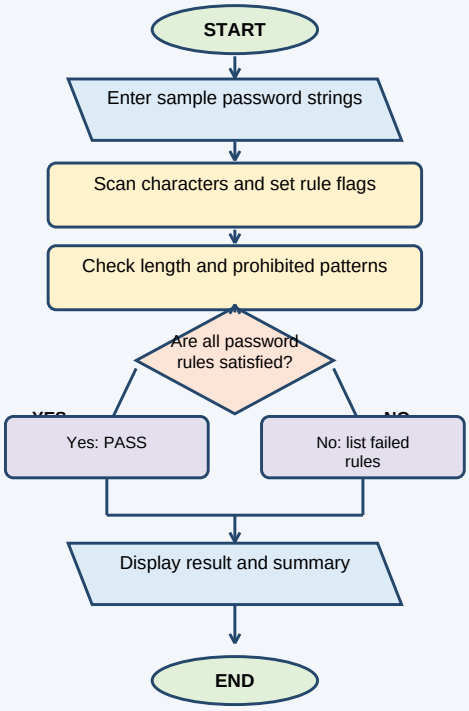
• Two identical adjacent characters are treated as a consecutive repetition.

Algorithm

1. Start and define the sample password strings.
2. For each password, initialise flags for uppercase, lowercase, digit, special character, and repetition.
3. Scan each character and update the flags.
4. Check the length and prohibited substring.
5. Build a list of all failed conditions.
6. Display PASS when no conditions fail; otherwise display FAIL and the failed rules.
7. Stop.

CASE 29 – FLOWCHART

Case 29: Password Policy Validation Engine



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 29 – PSEUDOCODE

```
BEGIN
    passwords = ["Python@123", "abc123", "Good#Pass8"]
    FOR each password IN passwords
        upper = FALSE; lower = FALSE; digit = FALSE; special = FALSE
        repeated = FALSE
        FOR i FROM 0 TO length(password)-1
            ch = password[i]
            IF ch is uppercase THEN upper = TRUE
            ELSE IF ch is lowercase THEN lower = TRUE
            ELSE IF ch is digit THEN digit = TRUE
            ELSE special = TRUE
            END IF
            IF i > 0 AND ch = password[i-1] THEN repeated = TRUE
        END FOR
        IF length(password) < 8 THEN PRINT "Length failed"
        IF NOT upper THEN PRINT "Uppercase failed"
        IF NOT lower THEN PRINT "Lowercase failed"
        IF NOT digit THEN PRINT "Digit failed"
        IF NOT special THEN PRINT "Special character failed"
        IF "password" occurs in lowercase(password) THEN PRINT "Prohibited word"
        IF repeated THEN PRINT "Repeated adjacent character"
    END FOR
END
```

Python Source Code

```
passwords = ["Python@123", "abc123", "Good#Pass8", "Passwoorrd9!"]

for password in passwords:
    has_upper = False
    has_lower = False
    has_digit = False
    has_special = False
    has_repeat = False
    failed = []

    for i in range(len(password)):
        ch = password[i]
        if ch.isupper():
            has_upper = True
        elif ch.islower():
            has_lower = True
        elif ch.isdigit():
            has_digit = True
        else:
            has_special = True

        if i > 0 and ch == password[i - 1]:
            has_repeat = True

    if len(password) < 8:
        failed.append("Minimum length 8")
    if not has_upper:
        failed.append("At least one uppercase letter")
    if not has_lower:
        failed.append("At least one lowercase letter")
    if not has_digit:
        failed.append("At least one digit")
    if not has_special:
        failed.append("At least one special character")
    if "password" in password.lower():
        failed.append("Prohibited substring")
    if has_repeat:
        failed.append("No consecutive repeated characters")

    print("\nPassword:", password)
    if len(failed) == 0:
        print("Result: PASS")
    else:
        print("Result: FAIL")
        for rule in failed:
            print("-", rule)
```

CASE 29 – TESTING AND RESULT

Test Cases

Sample password	Expected result	Reason
Python@123	PASS	Meets all listed rules
abc123	FAIL	Short; missing uppercase and special character
Good#Pass8	PASS	Meets all listed rules
Password9!	FAIL	Contains prohibited substring and adjacent repeated 's'

Sample Output

```
Password: Python@123
Result: PASS

Password: abc123
Result: FAIL
- Minimum length 8
- At least one uppercase letter
- At least one special character

Password: Good#Pass8
Result: PASS

Password: Password9!
Result: FAIL
- Prohibited substring
- No consecutive repeated characters
```

Conclusion

The program checks password samples with string methods, loops, Boolean variables, and conditional statements, and explains the policy rules that each sample fails.

CASE STUDY 31: String-Based Examination Answer Evaluation

Problem Statement

Write a Python program that compares students' answer strings with an answer key, counts correct, incorrect, and unanswered responses, applies negative marking, and calculates the final score.

Objectives

• Store the answer key and student responses as strings.

• Compare answers position by position.

• Count correct, incorrect, and unanswered responses.

• Apply positive and negative marks according to the stated marking scheme.

Assumptions

• Each question has one answer represented by a single character.

• The answer key and response string have equal lengths.

• A correct answer earns 1 mark, an incorrect answer loses 0.25 mark, and an unanswered question earns 0 marks.

• The sample contains five questions.

Algorithm

1. Start and define the answer key and response string.

2. Check that both strings have the same length.

3. Initialise correct, incorrect, unanswered, and score variables.

4. Compare each response character with the corresponding key character.

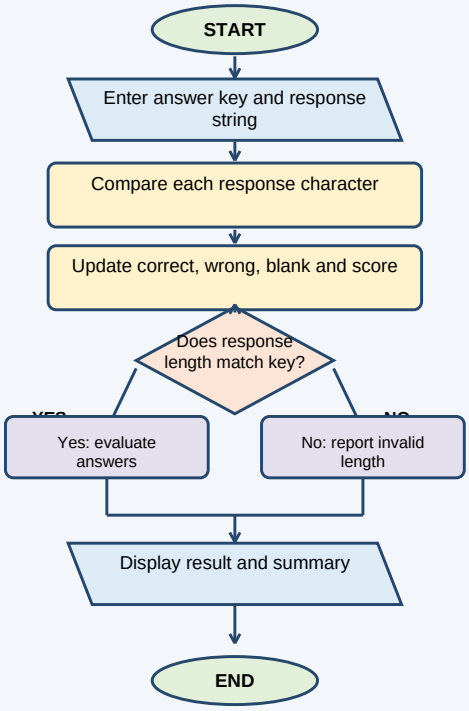
5. Update the appropriate count and score.

6. Display the counts and final score.

7. Stop.

CASE 31 – FLOWCHART

Case 31: String-Based Examination Answer Evaluation



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 31 – PSEUDOCODE

```
BEGIN
    key = "ABCDC"
    response = "ABX C"
    correct = 0
    incorrect = 0
    unanswered = 0
    score = 0
    IF length(key) != length(response) THEN
        PRINT "Invalid response length"
    ELSE
        FOR i FROM 0 TO length(key)-1
            IF response[i] = " " THEN
                unanswered = unanswered + 1
            ELSE IF response[i] = key[i] THEN
                correct = correct + 1
                score = score + 1
            ELSE
                incorrect = incorrect + 1
                score = score - 0.25
            END IF
        END FOR
        PRINT correct, incorrect, unanswered, score
    END IF
END
```

Python Source Code

```
answer_key = "ABCDC"
response = "ABX C"

if len(answer_key) != len(response):
    print("Invalid response length")
else:
    correct = 0
    incorrect = 0
    unanswered = 0
    score = 0

    for i in range(len(answer_key)):
        if response[i] == " ":
            unanswered += 1
        elif response[i].upper() == answer_key[i]:
            correct += 1
            score += 1
        else:
            incorrect += 1
            score -= 0.25

    print("Correct answers =", correct)
    print("Incorrect answers =", incorrect)
    print("Unanswered questions =", unanswered)
    print("Final score =", round(score, 2), "out of", len(answer_key))
```

CASE 31 – TESTING AND RESULT

Test Cases

Answer key	Response	Correct	Incorrect	Unanswered	Score
ABCDC	ABCDC	5	0	0	5.00
ABCDC	ABX C	3	1	1	2.75
ABCDC	XXXXX	0	5	0	-1.25
ABCDC	A C C	3	0	2	3.00

Sample Output

```
Correct answers = 3
Incorrect answers = 1
Unanswered questions = 1
Final score = 2.75 out of 5
```

Conclusion

The program demonstrates string indexing, comparison, loops, counters, and arithmetic operations to evaluate examination responses using a clearly defined marking policy.

CASE STUDY 35: Text Compression Using Character Runs

Problem Statement

Develop a Python program that compresses a string by grouping consecutive repeated characters, compares the original and encoded lengths, and keeps the original string when encoding does not reduce its length.

Objectives

• Read a text string from the user.

• Count consecutive occurrences of each character.

• Create a run-length encoded representation.

• Compare encoded and original lengths and select the shorter representation.

Assumptions

• The input is a non-empty string.

• Compression groups only consecutive identical characters.

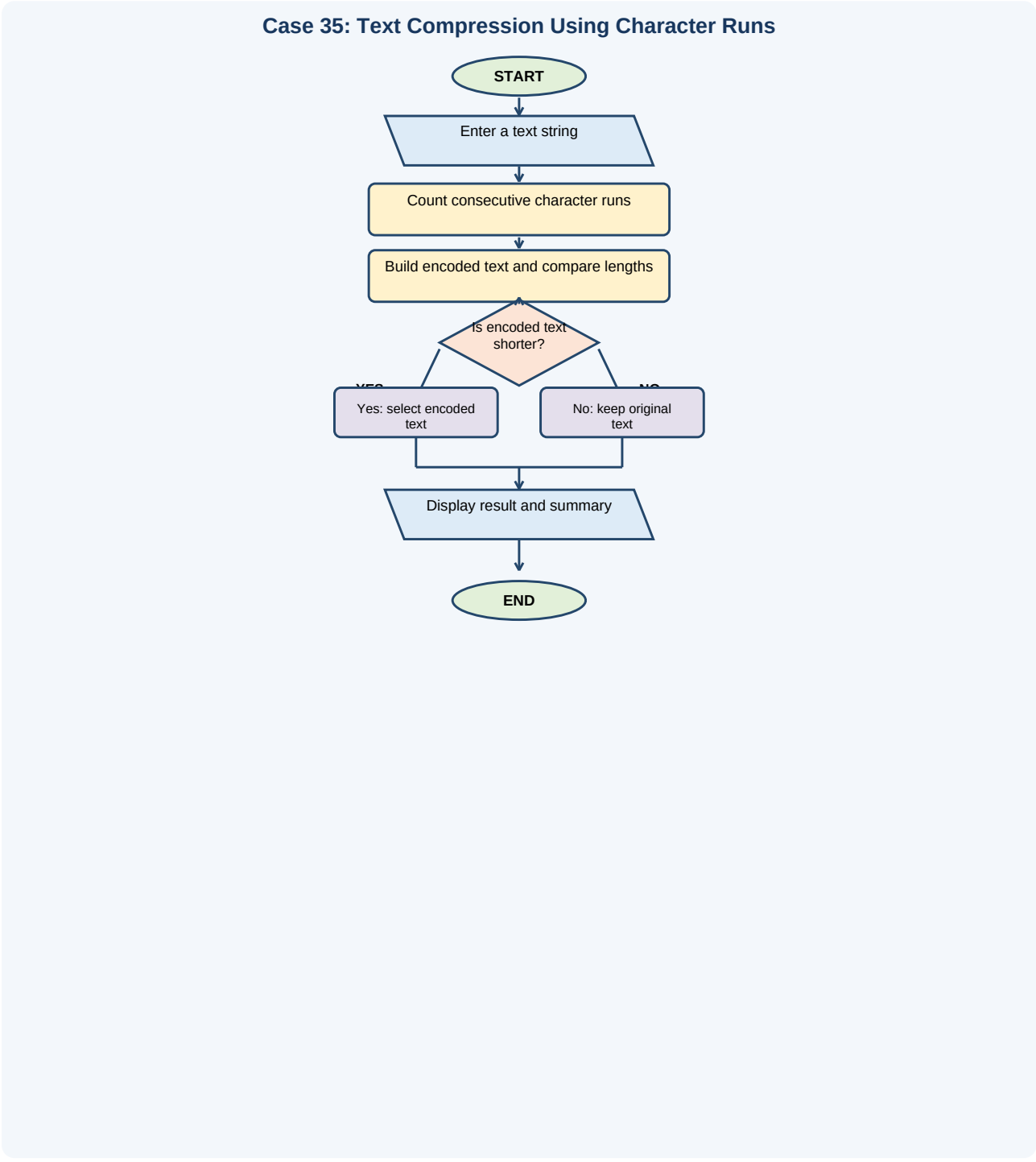
• Each run is represented by the character followed by its count.

• If the encoded string is not shorter, the original string is selected.

Algorithm

1. Start and input a string.
2. If the string is empty, display an appropriate message.
3. Set the current character and run count.
4. Scan the remaining characters; increase the count when the character matches.
5. When a different character is found, append the previous character and count to the encoded string.
6. Append the final run after the loop.
7. Compare encoded length with original length and select the shorter form.
8. Display both lengths and the selected representation.
9. Stop.

CASE 35 – FLOWCHART



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 35 – PSEUDOCODE

```
BEGIN
    INPUT text
    encoded = ""
    IF text is empty THEN
        PRINT "Empty input"
    ELSE
        current = text[0]
        count = 1
        FOR i FROM 1 TO length(text)-1
            IF text[i] = current THEN
                count = count + 1
            ELSE
                encoded = encoded + current + STRING(count)
                current = text[i]
                count = 1
            END IF
        END FOR
        encoded = encoded + current + STRING(count)
        IF length(encoded) < length(text) THEN
            result = encoded
        ELSE
            result = text
        END IF
        PRINT encoded, result
    END IF
END
```

Python Source Code

```
text = input("Enter text: ")

if text == "":
    print("Empty input")
else:
    encoded = ""
    current = text[0]
    count = 1

    for i in range(1, len(text)):
        if text[i] == current:
            count += 1
        else:
            encoded += current + str(count)
            current = text[i]
            count = 1

    encoded += current + str(count)

    print("Original text =", text)
    print("Encoded text =", encoded)
    print("Original length =", len(text))
    print("Encoded length =", len(encoded))

    if len(encoded) < len(text):
        result = encoded
        print("Selected representation: Encoded")
    else:
        result = text
        print("Selected representation: Original")

    print("Final representation =", result)
```

CASE 35 – TESTING AND RESULT

Test Cases

Input	Encoded form	Selected form
aaabbc	a3b2c1	Original: aaabbc
abcd	a1b1c1d1	abcd
zzzzzz	z6	z6
aabbaa	a2b2a2	aabbaa

Sample Output

```
Enter text: aaabbc
Original text = aaabbc
Encoded text = a3b2c1
Original length = 6
Encoded length = 6
Selected representation: Original
Final representation = aaabbc
```

Conclusion

The program uses string traversal and run counting to produce a simple run-length encoding, then chooses the representation with the shorter length.

CASE STUDY 42: Student Marks Matrix Analysis

Problem Statement

Create a Python program to analyse a matrix of student marks across multiple subjects. Calculate student totals and averages, subject averages, identify the top-performing student, and find subjects with low average marks.

Objectives

• Represent student marks using a nested list.

• Calculate row-wise totals and averages for students.

• Calculate column-wise averages for subjects.

• Identify the top performer and subjects below a defined average threshold.

Assumptions

• Each row represents one student and each column represents one subject.

• Marks are whole numbers from 0 to 100.

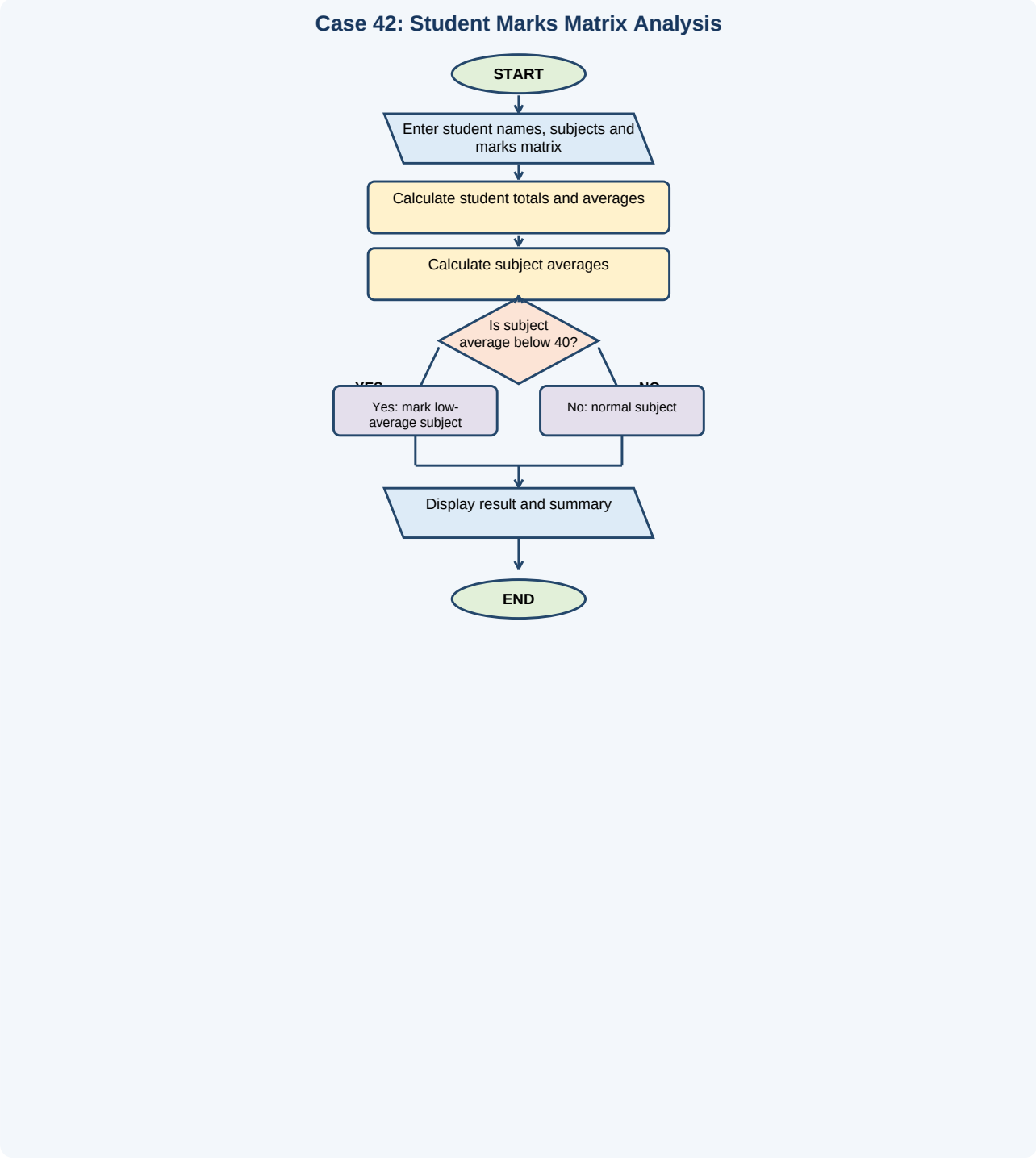
• The sample contains four students and three subjects.

• A subject average below 40 is considered low.

Algorithm

1. Start and define student names, subject names, and the marks matrix.
2. For each student row, calculate total and average marks.
3. Identify the student with the highest total.
4. For each subject column, calculate the average across all students.
5. Mark subjects whose average is below 40.
6. Display student-wise and subject-wise analysis and the top performer.
7. Stop.

CASE 42 – FLOWCHART



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 42 – PSEUDOCODE

```
BEGIN
    students = ["Asha", "Bala", "Charan", "Divya"]
    subjects = ["Python", "Maths", "English"]
    marks = [[80, 75, 85], [60, 65, 70],
             [35, 40, 45], [90, 88, 92]]
    top_total = -1
    FOR each student row i
        total = 0
        FOR each subject column j
            total = total + marks[i][j]
        END FOR
        average = total / number_of_subjects
        PRINT students[i], total, average
        IF total > top_total THEN
            top_total = total
            topper = students[i]
        END IF
    END FOR
    FOR each subject column j
        total = 0
        FOR each student row i
            total = total + marks[i][j]
        END FOR
        average = total / number_of_students
        PRINT subjects[j], average
        IF average < 40 THEN PRINT "Low average"
    END FOR
    PRINT topper
END
```

Python Source Code

```
students = ["Asha", "Bala", "Charan", "Divya"]
subjects = ["Python", "Maths", "English"]
marks = [
    [80, 75, 85],
    [60, 65, 70],
    [35, 40, 45],
    [90, 88, 92]
]

top_total = -1
topper = ""

print("STUDENT-WISE ANALYSIS")
for i in range(len(students)):
    total = 0
    for j in range(len(subjects)):
        total += marks[i][j]

    average = total / len(subjects)
    print(students[i], "Total:", total, "Average:", round(average, 2))

    if total > top_total:
        top_total = total
        topper = students[i]

print("\nSUBJECT-WISE AVERAGES")
for j in range(len(subjects)):
    subject_total = 0
    for i in range(len(students)):
        subject_total += marks[i][j]

    average = subject_total / len(students)
    print(subjects[j], "Average:", round(average, 2))
    if average < 40:
        print("Low average subject:", subjects[j])

print("\nTop performer:", topper, "with", top_total, "marks")
```

CASE 42 – TESTING AND RESULT

Test Cases

Student	Marks	Total	Average
Asha	80,75,85	240	80.00
Bala	60,65,70	195	65.00
Charan	35,40,45	120	40.00
Divya	90,88,92	270	90.00

Sample Output

```
STUDENT-WISE ANALYSIS
Asha Total: 240 Average: 80.0
Bala Total: 195 Average: 65.0
Charan Total: 120 Average: 40.0
Divya Total: 270 Average: 90.0

SUBJECT-WISE AVERAGES
Python Average: 66.25
Maths Average: 67.0
English Average: 73.0

Top performer: Divya with 270 marks
```

Conclusion

The program uses nested lists and nested loops to calculate row-wise and column-wise statistics and identify the top-performing student.

CASE STUDY 49: Tuple-Based Product Catalogue Audit

Problem Statement

Develop a Python program that stores product identifiers, prices, and stock quantities in tuples, validates catalogue records, calculates inventory values, and identifies products that need replenishment.

Objectives

• Represent each product record as a tuple.

• Validate product identifiers, prices, and stock quantities.

• Calculate the inventory value of each valid product.

• Identify products whose stock is below the replenishment level.

Assumptions

• Each product is stored as a tuple: (product_id, price, stock).

• Product identifiers must not be empty.

• Price and stock must be non-negative numbers; stock is a whole number.

• A product with stock below 5 units is marked for replenishment.

Algorithm

1. Start and define the product catalogue as a list of tuples.

2. Set the replenishment level to 5 and initialise the total inventory value.

3. Unpack each product tuple into identifier, price, and stock.

4. Validate the identifier, price, and stock values.

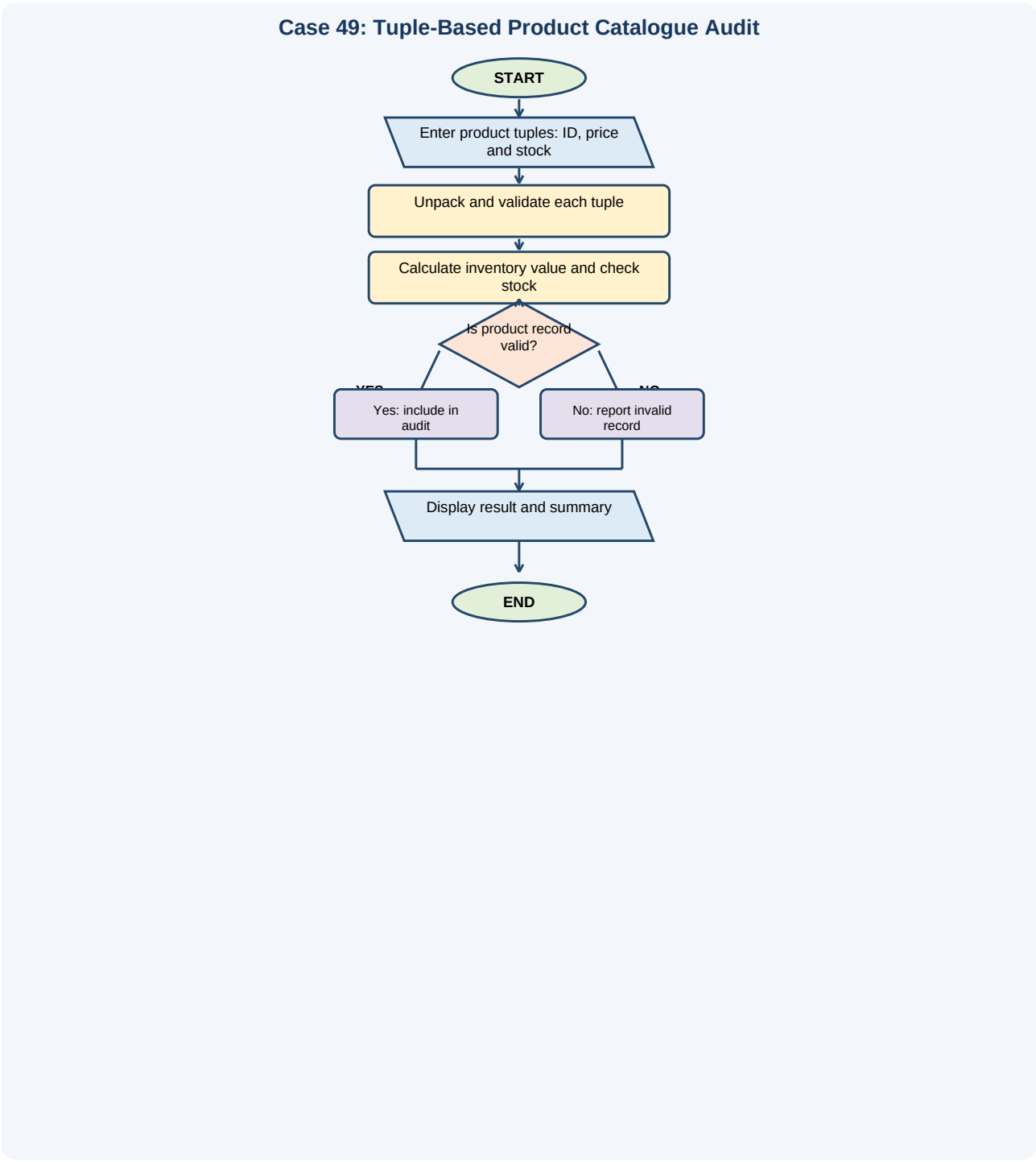
5. For each valid record, calculate price multiplied by stock and add it to the inventory value.

6. Flag valid products whose stock is below the replenishment level.

7. Display valid records, invalid records, replenishment alerts, and total inventory value.

8. Stop.

CASE 49 – FLOWCHART



Flowchart symbol key: Oval – Start/End; Parallelogram – Input/Output; Rectangle – Processing step; Diamond – Decision.

CASE 49 – PSEUDOCODE

```
BEGIN
    products = [("P101", 50, 10), ("P102", 120, 3),
                ("P103", 75, 8), ("", 40, 5)]
    reorder_level = 5
    inventory_value = 0
    FOR each product IN products
        id, price, stock = product
        IF id = "" OR price < 0 OR stock < 0 THEN
            PRINT "Invalid record", product
        ELSE
            value = price * stock
            inventory_value = inventory_value + value
            PRINT id, value
            IF stock < reorder_level THEN
                PRINT "Replenishment required", id
            END IF
        END IF
    END FOR
    PRINT inventory_value
END
```

Python Source Code

```
products = [
    ("P101", 50, 10),
    ("P102", 120, 3),
    ("P103", 75, 8),
    ("", 40, 5),
    ("P105", 30, 0)
]

reorder_level = 5
inventory_value = 0

print("PRODUCT CATALOGUE AUDIT")
for product in products:
    product_id, price, stock = product

    if product_id == "" or price < 0 or stock < 0:
        print("Invalid record:", product)
    else:
        value = price * stock
        inventory_value += value
        print(product_id, "Inventory value = ■", value)

        if stock < reorder_level:
            print("Replenishment required for", product_id)

print("Total inventory value = ■", inventory_value)
```

CASE 49 – TESTING AND RESULT

Test Cases

Product tuple	Expected validation	Inventory value	Reorder alert
('P101',50,10)	Valid	■500	No
('P102',120,3)	Valid	■360	Yes
('P103',75,8)	Valid	■600	No
('',40,5)	Invalid: empty ID	Not included	No
('P105',30,0)	Valid	■0	Yes

Sample Output

```
PRODUCT CATALOGUE AUDIT
P101 Inventory value = ■ 500
P102 Inventory value = ■ 360
Replenishment required for P102
P103 Inventory value = ■ 600
Invalid record: ('', 40, 5)
P105 Inventory value = ■ 0
Replenishment required for P105
Total inventory value = ■ 1460
```

Conclusion

The program demonstrates tuples, tuple unpacking, lists, loops, and conditional statements to validate product records and audit stock value and replenishment needs.